

Assignment 3

Section-A

1. Explain the primary objective of unit testing in software development.

→ Unit testing involves the testing of each unit or an individual component of the software application. It is the first level of functional testing. The aim behind unit testing is to validate unit components with its performance. A unit is a single testable part of a software and tested during the development phase of the application software. The purpose of unit testing is to test the correctness of isolated code. A unit component is an individual function or code of the application.

2. Differentiate between validation and verification in software testing.

→ **Verification:** It is the process of checking that a software achieves its goals without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements we have. It is also known as static testing. Verification means "Are we building the product right?"

Validation: It is the process of checking whether the software product is up to the mark or in other words product has high level requirements. It is the process of checking the validation of product i.e. it checks what we are developing is the right product. It is also known as dynamic testing. Validation means "Are we building the right product?"

3. What is the purpose of regression testing? Provide an example scenario where regression testing is essential.

→ Regression testing is used to authenticate a code change in the software does not impact the existing functionality of the product. Regression testing is making sure that the products works fine with new functionality, bug fixes, or any change in the existing feature.

Regression testing is a type of software testing. Test cases are re-executed to check the previous functionality of the application is working fine and new changes have not produced any bugs. Example - Fixing password validation for special characters, ensuring no issues & confirming successful registration.

4. Define Black-Box testing and white box testing. Highlight one advantage and one limitation of each approach.

→ Black-Box testing is a technique of software testing which examines the functionality of software without peering into its internal structure or coding.

Advantage: It is efficient for implementing the tests in larger system.

Limitation: There is a possibility of repeating the same tests while implementing the testing process.

White box testing techniques analyze the internal structures, internal design, code structure and the working of the software rather than just the functionality as in black box testing. White Box testing also known as open box testing or transparent testing.

Advantage: White Box testing is thorough as the entire code and structures are tested.

Limitation: It is very expensive.

5. Name two CASE (Computer-Aided Software Engineering) tools commonly used in software development. Provide a

brief description of one of them.

- Two case tools are: (i) Diagramming Tools
(ii) Analysis Tools

Diagramming tools: It helps in diagrammatic and graphical representations of the data and system processes. It represents system elements, control flow and data flow among different software components and system structures in a pictorial form. For example - Flow chart Maker tool.

Section - B

1. Discuss the objectives of integration testing in the software development life cycle. Provide an example of an integration testing scenario.

→ The objectives of integration testing in the software development life cycle:

- (i) Verify the interactions between components: Ensures that the interfaces between different components/modules are working as expected.
- (ii) Ensure Compatibility: Confirm that the integrated components are compatible with each other and with external systems or services they interact with.
- (iii) Detect problems early: Identify and address defects related to the interaction between integrated components.
- (iv) Improve the overall reliability of the system.
- (v) Improve the quality of the system by identifying and fixing issues before they become more difficult and expensive to resolve.

Example of an integration testing scenario:

Let us assume that we have a Gmail application where

we perform the integration testing. First we will do functional testing on the login page, which includes various components such as username, password, submit and cancel button. Then only we can perform integration testing. The different integration scenarios as follows:

Scenario 1:

- First we login as P user and click on compose mail and performing the functional testing for the specific components.
- Now we click on the send and also ~~it~~ check for save drafts.
- After that, we send a mail to Q and verify in the send items folder of P to check if the send mail is there.
- Now, we will log out as P and login as Q and move to the Inbox and verify if the mail has reached.

Scenario 2: We also perform the integration testing on Spam folders.

2. Compare and Contrast dynamic analysis and Static analysis in the context of software testing. Highlight their strength and weakness.

- **Dynamic Analysis:** Dynamic Analysis is a type of software testing which is performed to analyze the dynamic behaviour of the code. It includes the testing of the software for the input values and output values that are analyzed.

Strength:

- (i) **Real-World Scenarios:** Dynamic analysis allows testers to observe the software behaviour in real world scenarios.
- (ii) It identifies vulnerabilities in a run time environment.
- (iii) It allows for analysis of applications in which you do not have access to the actual code.

Weakness:

- (i) Automated tools provide false positives and false negatives
- (ii) It is more difficult to trace the vulnerability back to the exact location in the code, taking longer to fix the problem.

Static Analysis: It is a type of a software testing method which is performed to check the defects in software without actually executing the code of the software application. The errors that can't be found using Dynamic analysis, can be easily found by static Testing.

Strength:

- (i) It can find weakness in the code at the exact location.
- (ii) Weakness are found earlier in the development life cycle, reducing the cost to fix.

Weakness:

- (i) Time consuming if conducted manually.
- (ii) It does not found vulnerabilities introduced in the runtime environment.

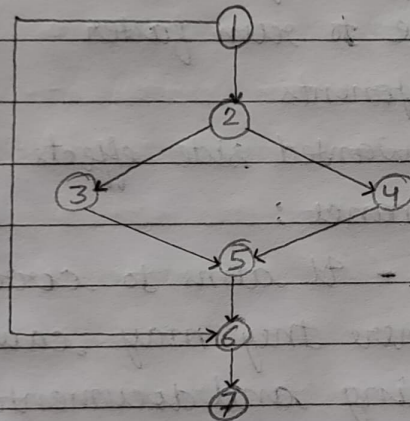
3. Describe the importance of Control Flow Graphs (CFGs) in software testing. Provide a step by step example of constructing a CFG for a simple code snippet.

→ A Control flow Graph (CFG) is the graphical representation of control flow or computation during the execution of the program or applications. Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside of a program unit. CFG is a process oriented and shows all the paths that can be traversed during a program execution. It is a directed graph. Edges in

CFGs portray control flow paths and the nodes in. CFGs portray basic block blocks.

Example: If $A = 10$ then
 if $B > C$
 $A = B$
 else $A = C$
 end if
 end if.
 print A, B, C

Control Flow Graph:



Section-C

1. Software Maintenance is a critical phase in the software development life cycle. Discuss the reasons why software maintenance is essential and elaborate types of maintenance activities involved, providing examples for each.

→ Software Maintenance: It refers to the process of modifying and updating a software system after it has been delivered to the customer. This can include fixing bugs, adding new features, improving performance or updating the software to work with new hardware or software systems.

Software Maintenance is a critical part of the Software

development life cycle (SDLC) and is necessary to ensure that the software continues to meet the needs of the users over time. It is also important to consider the cost and effort required for software maintenance when planning and developing a software system. Software Maintenance is essential for:

- Correct errors
- Change in user requirement with time.
- Changing hardware/software requirements
- To improve system efficiency
- To optimize the code to run faster
- To modify the components
- To reduce any unwanted side effects

Types of Software Maintenance :

1. Corrective Maintenance - It aims to correct any remaining errors regardless of where they may cause specifications, design, coding, testing and documentation, etc.

Example: ~~Modifying~~ Patching a software application to address a critical security vulnerability.

2. Adaptive Maintenance: It contains only modifying the software to match changes in the ever-changing environment.

Ex: Modifying a web application to support a new version of web browser.

3. Preventive Maintenance: It is the process by which we prevent our system from being obsolete. It involves the concept of reengineering and reverse engineering in which an old system with old technology is re-engineered using new technology.

Example: Conducting routine code reviews and refactoring to improve code maintainability and reduce the likelihood of future bugs.

4. Perfective Maintenance: It defines improving processing efficiency or performance or restricting the software to enhance changeability. This may contain enhancement of existing system functionality, improvement in computational efficiency etc.

Example: Enhancing the user interface of a software application for a better user experience.

2. Elaborate on the principles and techniques of white-box testing. Describe how code coverage metrics, such as statement coverage and branch coverage, play a role in white box testing.

→ White Box Testing is also known as glass box testing, structural testing, clear box testing, open box testing and transparent box testing. It tests the internal coding and infrastructure of a software focus on checking of predefined inputs against expected and desired outputs. It is based on inner workings of an application and revolves around internal structure testing.

Working principle of white box testing:

- Input: Requirements, Functional specifications, design documents, code.
- Processing: Performing risk analysis to guide through entire process.
- Proper test planning: Designing test cases so as to cover the entire code. Execute suite - repeat until error free software is reached. Also, the results are communicated.
- Output: Preparing final report of the entire testing process.

Techniques of White Box Testing :

- (i) Statement Coverage : In this technique, the aim is to traverse all the statements at least once. Hence, each line of code is tested. Measures the percentage of executable
$$\text{Statement Coverage} = \left(\frac{\text{Number of executed statements}}{\text{Total number of statements in source code}} \right) \times 100$$
- (ii) Branch Coverage : In this, Test cases are designed so that each branch from all decision points is traversed at least once. Measures the percentage of decision branches that have been exercised during testing.
- (iii) Condition Coverage : Focuses on testing different conditions within a decision statement. Ensures that all the possible ~~conditions~~ combinations of conditions within a decision statement are tested, helping identify conditions that may lead to errors.
- (iv) Path Testing : In this, control flow graphs are made from code ~~to~~ or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each path.
- (v) Loop Testing : Loops are widely used and these are fundamental to many algorithms hence, their testing is very important. Errors often occur at the beginning and ends of loops.

Code Coverage metrics, such as statement and branch coverage, play a pivotal role in white-box testing by quantifying the extent of code exercised during testing. Statement coverage measures the percentage of executable code lines tested, providing insights into the completeness of test cases for individual code statements.

Branch coverage, on the other hand, assesses decision branches in conditional statements, ensuring that both true and false branches are executed. Overall, these metrics contribute to white-box testing by evaluating test completeness and ultimately enhancing software quality and reliability.

3. Explain the concept of static analysis in software testing. Provide an example of how static analysis can help detect potential issues in code.

→ Static Analysis in software testing is a method of examining software code without executing the program. It involves analyzing the source code, design and documentation to find potential issues, security vulnerabilities or code quality problems before the code is actually run. The goal is to identify and fix these issues early in the development process, reducing the likelihood of bugs and enhancing the overall quality of the software.

Example:

Consider the following Python code snippet:

```
def divide(a,b):  
    result = a/b  
    return result
```

Potential issues identified by static analysis:

- (i) Division by zero: If b is zero, the code will result in a runtime error. Static analysis can identify this potential issue without executing the code.
- (ii) Variable type: The code does not specify the types of a and b . Static analysis tools can check for type mismatches or undefined variables.

(iii) Error handling: The code lacks error-handling mechanisms for potential exceptions during the division operation.

Now static code Analysis Helps:

(i) Division By zero Detection

(ii) Type Checking

(iii) Error handling suggestion

Updated Code after static Analysis suggestion:

```
def divide(a,b):
```

```
    if b!=0:
```

```
        result = a/b
```

```
        return result
```

```
    else:
```

```
        print("Error: division by zero not allowed.")
```

```
        return None
```